UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"

Campus de Sorocaba
ICTS Instituto de Ciências e Tecnologia

Trabalho 2

Redes de Comunicação Industrial

Discente: Luiz Henrique Corrêa Monteiro, Paulo de Melo Pereira R. Ramalho
Orientador: Prof. Dr. Eduardo Godoy Paciência

Sorocaba - SP
2026

Luiz Henrique Corrêa Monteiro, Paulo de Melo Pereira R. Ramalho

Programação Remota, Comunicação e Visualização Web usando API Rest HTTP

Trabalho apresentado ao Instituto de Ciências e Tecnologia da Unesp, como parte dos requisitos para a conclusão da disciplina em Redes de Comunicação Industrial.

Orientador: Prof. Dr. Eduardo Godoy Paciência

E-mail dos alunos autores:
lh.monteiro@unesp.br
paulo.ramalho@unesp.br

Sorocaba - SP
2026

Índice

1 Introdução **2**

2 Dificuldades e impressões **2**

3 Recursos utilizados **4**
3.1 Descrição do módulo 4
3.2 Descrição da arquitetura de comunicação 4
3.3 Metodologia de implementação 5

4 Programação **6**
4.1 Programação do ESP32 6
4.2 Comunicação através da API REST 8
4.3 Construção da interface Node-RED 9
4.3.1 Entrada dos dados 10
4.3.2 Exposição dos dados obtidos 10
4.3.3 Status de conexão 11
4.3.4 Dashboard construído 11
4.4 Realização do OTA 12
4.5 Desenvolvimentos extras 13
4.5.1 Simulador do ESP32 13
4.5.2 Interface Web 14
4.5.3 Tela de LOGS no Node-RED 15
4.5.4 Banco de dados PostgreSQL 15

5 Conclusão **16**

1 Introdução

Atualmente, sistemas de automação e monitoramento utilizam cada vez mais a comunicação em rede para permitir a troca de informações entre dispositivos físicos e aplicações de supervisão. Dentro desse contexto, uma das formas de realizar essa comunicação é através do protocolo HTTP associado a uma API REST, permitindo que diferentes dispositivos realizem operações de envio, consulta e alteração de informações através de uma rede.

Neste trabalho foi desenvolvido um sistema de comunicação entre uma placa ESP32 e uma interface de supervisão construída no Node-RED. O objetivo principal foi realizar a aquisição de variáveis ambientais utilizando um sensor BME280 e disponibilizar essas informações através de uma API REST. Além da aquisição dos sensores, também foi realizada a leitura de um botão físico e o controle remoto de um LED conectado ao ESP32.

O sensor BME280 permite realizar medições de temperatura, umidade relativa e pressão atmosférica. A partir da pressão também é possível realizar uma estimativa da altitude, utilizando como referência um valor conhecido de pressão ao nível do mar. Essas informações são adquiridas pelo ESP32 e posteriormente disponibilizadas para utilização pelo sistema.

A comunicação entre os diferentes elementos foi realizada através de uma rede Wi-Fi utilizando requisições HTTP. A API REST permite realizar a troca de informações entre o dispositivo e o Node-RED, fazendo com que os dados adquiridos pelo ESP32 possam ser apresentados em um dashboard e, no sentido contrário, que comandos realizados através da interface possam alterar o estado do LED.

O Node-RED foi utilizado para construção da interface principal de supervisão. No dashboard desenvolvido é possível acompanhar os valores provenientes do BME280, verificar o estado do botão, visualizar o estado do LED e realizar seu acionamento remotamente. Dessa maneira, foi possível implementar tanto o monitoramento quanto a atuação através da comunicação REST.

Outro requisito desenvolvido durante o trabalho foi a programação remota do ESP32 utilizando OTA (*Over-the-Air*). Após uma primeira programação realizada através da conexão física, o recurso permite enviar novas versões do firmware pela rede Wi-Fi, reduzindo a necessidade de acesso físico ao dispositivo durante atualizações posteriores.

Além dos requisitos principais do trabalho, também foram desenvolvidas algumas funcionalidades extras, que são apresentadas a seguir. Dessa forma, o desenvolvimento principal do trabalho está concentrado na integração entre ESP32, sensor BME280, API REST e Node-RED, juntamente com a implementação da atualização remota através de OTA. Os demais recursos desenvolvidos são apresentados separadamente como funcionalidades adicionais.

mas distribuídos e conectados.

2 Dificuldades e impressões

Durante o desenvolvimento do trabalho foram encontradas algumas dificuldades relacionadas principalmente à integração entre os diferentes componentes. Individualmente, o funcionamento do simulador, backend, banco de dados e interfaces era relativamente simples de verificar. Entretanto, quando todos os módulos passaram a trabalhar simultaneamente, pequenas diferenças de configuração começaram a causar comportamentos incorretos.

Uma das primeiras dificuldades ocorreu durante a implementação da seleção entre o simulador e o ESP32 real. Inicialmente, alterar a opção apresentada na interface não significava necessariamente que o backend estava utilizando a mesma fonte. Dessa forma, em algumas situações o dashboard indicava que o ESP32 real estava selecionado enquanto a API ainda considerava o simulador como dispositivo ativo.

Para solucionar esse problema foi criada uma lógica específica de seleção da fonte através da API. A partir dessa alteração, as interfaces passaram a consultar e modificar a fonte ativa diretamente no backend. Foram utilizados identificadores diferentes para cada dispositivo, permitindo distinguir `esp32-001` de `esp32-simulator-001`.

Essa alteração também resultou em um mecanismo de proteção contra dados provenientes do dispositivo incorreto. Quando o backend recebe determinadas requisições de um dispositivo diferente daquele atualmente selecionado, a operação pode ser rejeitada. Durante os testes esse comportamento apareceu através de respostas HTTP de conflito, o que inicialmente parecia indicar uma falha de comunicação, mas na realidade mostrava que a seleção de fonte estava funcionando.

Outra dificuldade encontrada esteve relacionada aos endereços utilizados para comunicação com a API. Durante o desenvolvimento no próprio computador, alguns testes foram realizados utilizando o endereço local `127.0.0.1`. Entretanto, esse endereço representa a própria máquina que realiza a requisição e não pode ser utilizado pelo ESP32 para acessar o servidor executado em outro equipamento.

Foi necessário, portanto, configurar o ESP32 e as demais aplicações utilizando o endereço de rede correspondente ao servidor. A padronização desses endereços foi importante para evitar situações em que a interface Web funcionava corretamente no computador, mas o dispositivo físico não conseguia alcançar a API.

A integração com o ESP32 real também apresentou dificuldades que não apareciam durante a utilização do simulador. Enquanto o simulador dependia apenas dos programas executados no computador, o dispositivo físico dependia também da conexão Wi-Fi, intensidade do sinal, configuração correta do endereço do servidor e disponibilidade da API.

Durante essa etapa, a análise das respostas HTTP fornecidas pelo servidor foi importante para diferenciar problemas de conexão de erros relacionados à lógica da aplicação. A observação das requisições recebidas pelo backend também permitiu verificar se o ESP32 estava enviando telemetria e consultando corretamente o estado do LED.

A implementação do OTA apresentou outra dificuldade durante o desenvolvimento. Apesar de o dispositivo estar conectado à rede e executar corretamente o firmware, a identificação automática através do nome configurado depende do funcionamento adequado dos serviços de descoberta na rede. Em algumas situações o dispositivo não aparecia imediatamente como uma opção disponível para programação.

Após a configuração e os testes necessários, foi possível realizar a atualização do firmware pela rede. Essa funcionalidade facilitou consideravelmente as alterações posteriores, pois eliminou a necessidade de manter o dispositivo permanentemente conectado através do cabo utilizado para programação.

Também foi possível observar a vantagem da utilização de uma API como ponto central do sistema. Durante o desenvolvimento foram utilizadas duas interfaces diferentes, Node-RED e uma aplicação Web, além de duas fontes de dados, o simulador e o ESP32. Mesmo assim, não foi necessário desenvolver uma estrutura de comunicação completamente diferente para cada combinação, pois todos os módulos utilizavam a API como elemento intermediário.

De maneira geral, as maiores dificuldades não estiveram na leitura individual do sensor ou no acionamento do LED, mas na integração e sincronização de todos os elementos. Essa etapa permitiu compreender melhor problemas que aparecem em sistemas distribuídos, principalmente aqueles relacionados à identificação dos dispositivos, endereçamento de rede, disponibilidade dos serviços e consistência das informações apresentadas ao usuário.

3 Recursos utilizados

Para o desenvolvimento do sistema foram utilizados recursos de hardware e software responsáveis pela aquisição das variáveis, comunicação através da rede, visualização das informações e atualização remota do firmware. Entre os principais elementos utilizados estão a placa ESP32, o sensor BME280, o protocolo HTTP, a API REST e a plataforma Node-RED.

3.1 Descrição do módulo

O elemento principal de hardware utilizado no projeto foi uma placa baseada no microcontrolador ESP32. A conectividade Wi-Fi disponível no dispositivo foi utilizada para estabelecer a comunicação com os demais elementos do sistema através da rede.

Além do microcontrolador, o módulo utilizado possui um display OLED, empregado durante o desenvolvimento para apresentar localmente algumas informações relacionadas ao funcionamento do sistema. Dessa forma, foi possível observar diretamente no dispositivo os valores adquiridos e algumas informações relacionadas à conexão.

Para aquisição das variáveis ambientais foi utilizado o sensor Bosch BME280. O sensor permite realizar medições de temperatura, umidade relativa e pressão atmosférica, sendo conectado ao ESP32 através do barramento I^2C.

A altitude também é apresentada pelo dispositivo, porém esse valor não corresponde a uma medição direta realizada pelo BME280. Seu valor é estimado através da pressão atmosférica medida e de uma pressão de referência ao nível do mar.

Além do BME280, foi utilizado um botão como entrada digital e um LED como saída. O botão permite enviar para o sistema uma informação proveniente de uma entrada física, enquanto o LED permite verificar o funcionamento da comunicação no sentido contrário, uma vez que seu estado pode ser alterado remotamente.

A Figura 1 apresenta o diagrama de pinos utilizado como referência durante a implementação.

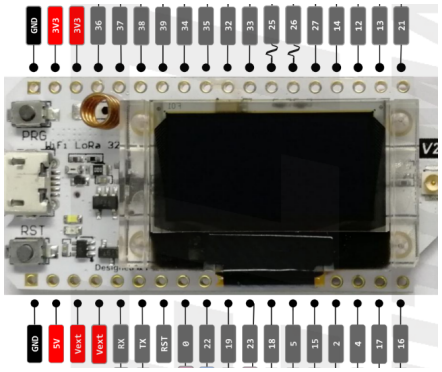


Figura 1: Diagrama de pinos do módulo ESP32 utilizado no projeto.

3.2 Descrição da arquitetura de comunicação

A arquitetura principal do trabalho foi desenvolvida com o objetivo de permitir a comunicação entre o ESP32 e o Node-RED através de uma API REST utilizando o protocolo HTTP.

O ESP32 é responsável pela aquisição das variáveis físicas. Periodicamente são realizadas leituras do sensor BME280 e do estado do botão. Essas informações são então organizadas pelo firmware e enviadas através da rede.

A API REST é utilizada como meio de comunicação entre os diferentes elementos do sistema. Através de requisições HTTP, os dados adquiridos pelo ESP32 podem ser disponibilizados para o Node-RED, onde são posteriormente tratados e apresentados ao usuário.

De maneira simplificada, o fluxo relacionado à aquisição das informações pode ser representado por:

BME280 / Botão → ESP32 → API REST → Node-RED

Além da aquisição, foi implementada uma comunicação no sentido contrário para permitir a atuação remota sobre o LED. Nesse caso, o usuário realiza uma alteração através do dashboard do Node-RED e o comando correspondente é disponibilizado através da API para o ESP32.

O fluxo de atuação pode ser representado por:

Node-RED → API REST → ESP32 → LED

Essa estrutura permite demonstrar duas situações diferentes de comunicação. Na primeira, o ESP32 atua como origem das informações adquiridas pelos sensores. Na segunda, o dispositivo recebe uma informação proveniente da interface e realiza uma ação física.

A utilização de uma API REST permite separar a aquisição realizada pelo dispositivo da apresentação realizada no Node-RED. Dessa maneira, a comunicação ocorre através de rotas e mensagens definidas, sem a necessidade de uma ligação direta entre os elementos da interface e cada função implementada no firmware.

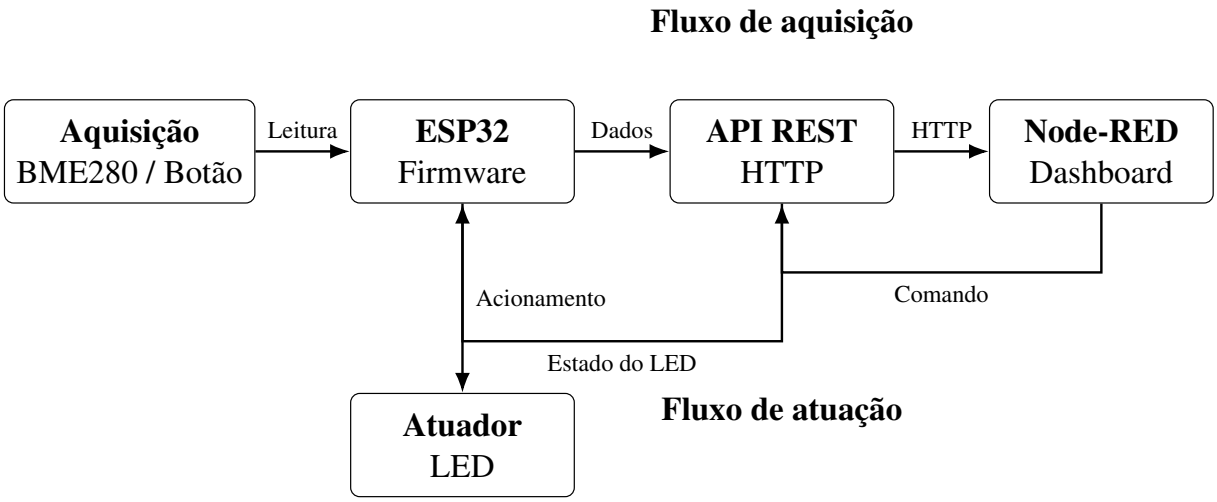


Figura 2: Arquitetura principal de comunicação entre o ESP32, a API REST e o Node-RED.

3.3 Metodologia de implementação

A implementação foi realizada inicialmente através do desenvolvimento da comunicação necessária entre o ESP32 e a API REST. O firmware foi preparado para realizar a leitura do sensor BME280 e do botão, além de permitir o acionamento do LED.

Após a leitura correta das variáveis, foi realizada a implementação das requisições HTTP necessárias para a troca de informações. Nessa etapa foram realizados testes para verificar o envio da telemetria e a consulta do estado do atuador.

Em seguida foi desenvolvido o fluxo no Node-RED. A interface passou a consultar as informações disponibilizadas pela API e apresentar os valores de temperatura, umidade e pressão, além dos estados correspondentes ao botão e ao LED.

Posteriormente foi implementado o caminho contrário da comunicação, permitindo que o estado do LED fosse alterado através do dashboard. Dessa forma, foi possível verificar a comunicação tanto no sentido ESP32 para Node-RED quanto no sentido Nodered para ESP32.

Por último, foi configurada a atualização remota do firmware utilizando OTA. Após a programação inicial através da conexão física, foram realizados testes de atualização utilizando a rede Wi-Fi.

Após o funcionamento dos requisitos principais, foram desenvolvidas separadamente algumas funcionalidades adicionais, apresentadas posteriormente na subseção de desenvolvimento extra.

4 Programação

O desenvolvimento do sistema envolveu principalmente três partes: a programação do ESP32, a implementação da comunicação através da API REST e a construção da interface de supervisão utilizando Node-RED. Além disso, foi implementada a atualização remota do firmware utilizando OTA.

Para facilitar a organização dos diferentes componentes desenvolvidos, o repositório foi dividido em diretórios específicos para cada parte do sistema. Sua estrutura principal é apresentada a seguir:

Trabalho_02_RIC

```
Trabalho\_02\_RIC/
|
+-- Code/
|   |
|   +-- backend/
|   |
|   +-- firmware/          # Código do ESP32 (.ino)
|   |
|   +-- frontend/
|   |
|   +-- simulator/
|   |
|   +-- README.md
|
+-- Node\_red/
|   |
|   +-- Node\_red.json
|
+-- ...
```

O firmware utilizado no ESP32 encontra-se no diretório `Code/firmware/`, enquanto o fluxo desenvolvido no Node-RED está disponível em `Node_red/Node_red.json`. Os demais diretórios presentes em `Code/` estão relacionados aos recursos adicionais implementados no projeto, incluindo a API, o simulador e a interface web.

4.1 Programação do ESP32

O firmware do ESP32 foi desenvolvido em C++ utilizando o framework Arduino. O programa é responsável pela leitura do BME280, leitura do botão, controle do LED, atualização do display OLED, conexão com a rede Wi-Fi, comunicação HTTP e funcionamento do OTA.

Para implementação disso, foram utilizadas bibliotecas específicas para comunicação Wi-Fi e HTTP, além daquelas em uso para o BME280, display OLED e atualização OTA.

O sensor BME280 foi conectado através do barramento I^2C. Durante a inicialização do dispositivo, o firmware tenta localizar o sensor inicialmente através do endereço `0x76`. Caso não seja encontrado, também é realizada uma tentativa utilizando o endereço `0x77`.

Após a inicialização, são realizadas leituras periódicas de temperatura, umidade e pressão atmosférica.

O botão foi configurado utilizando uma entrada digital com `INPUT_PULLUP`. Dessa forma, o estado lógico do pino é alterado quando ocorre o acionamento físico do botão. Essa informação é enviada juntamente com os demais dados adquiridos pelo ESP32.

O LED foi configurado como saída digital e representa o atuador utilizado no projeto. Diferentemente das informações do BME280, que seguem do ESP32 para o sistema de supervisão, o estado desejado para o LED é recebido através da comunicação com a API.

Para facilitar o acompanhamento do dispositivo durante os testes, também foi utilizado o display OLED presente no módulo. Nele são apresentados os valores obtidos pelo BME280, o endereço IP do ESP32 e os estados do botão e do LED.

A comunicação com a rede é realizada através do Wi-Fi. Durante a inicialização, o ESP32 tenta estabelecer conexão com a rede configurada e, após a conexão, passa a realizar as requisições HTTP necessárias para funcionamento do sistema.

O firmware também possui uma rotina de reconexão. Caso a conexão Wi-Fi seja interrompida durante a execução, novas tentativas são realizadas sem que seja necessário reinicializar manualmente o dispositivo.

Para evitar uma quantidade excessiva de requisições, foram definidos intervalos para cada operação. O envio da telemetria é realizado aproximadamente a cada cinco segundos, enquanto a consulta do estado do LED ocorre aproximadamente a cada dois segundos.

A Figura 3 apresenta o dispositivo durante os testes com o LED desligado.

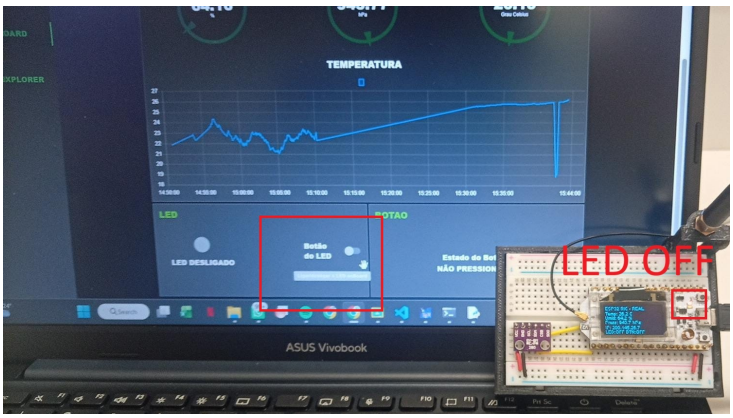


Figura 3: ESP32 durante os testes com o LED desligado.

A Figura 4 apresenta o mesmo dispositivo após o acionamento do LED.

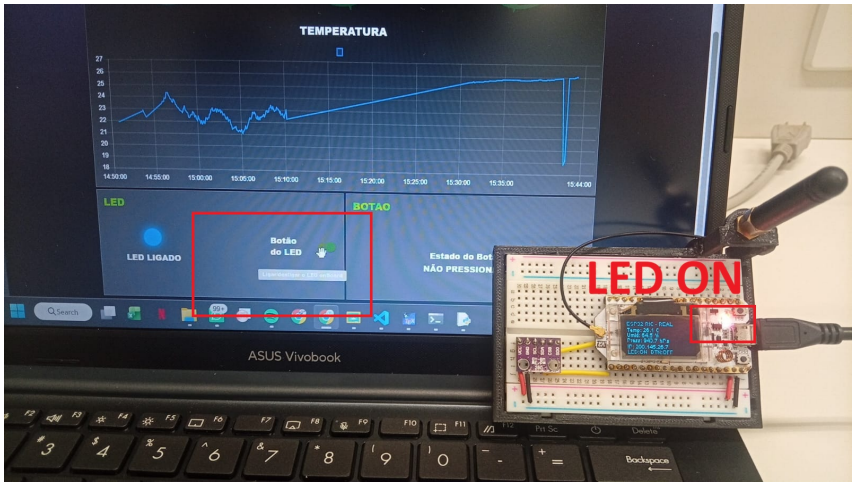


Figura 4: ESP32 durante os testes com o LED acionado.

4.2 Comunicação através da API REST

A comunicação entre o ESP32 e o Node-RED foi realizada utilizando uma API REST. Essa estrutura permite organizar diferentes operações através de rotas HTTP, separando o envio das informações dos sensores das operações relacionadas ao atuador.

No envio da telemetria, o ESP32 realiza uma requisição contendo as informações adquiridas. Entre os dados enviados estão o identificador do dispositivo, temperatura, umidade, pressão e estado do botão.

Para o controle do LED é utilizado o processo contrário. O estado solicitado pelo usuário através do Node-RED é disponibilizado pela API e consultado periodicamente pelo ESP32. Após receber a informação, o firmware atualiza o estado da saída correspondente.

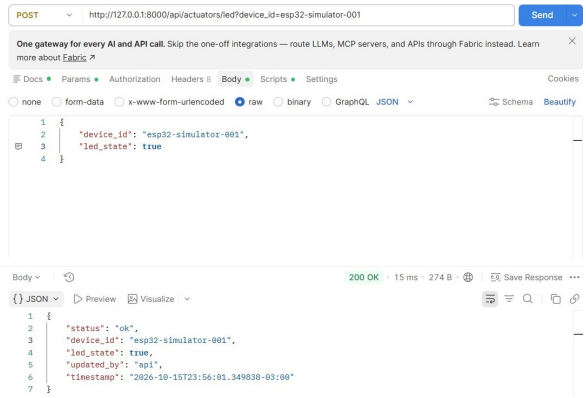
Entre as rotas utilizadas durante o funcionamento do sistema estão:

Tabela 1: Principais rotas utilizadas na comunicação REST.

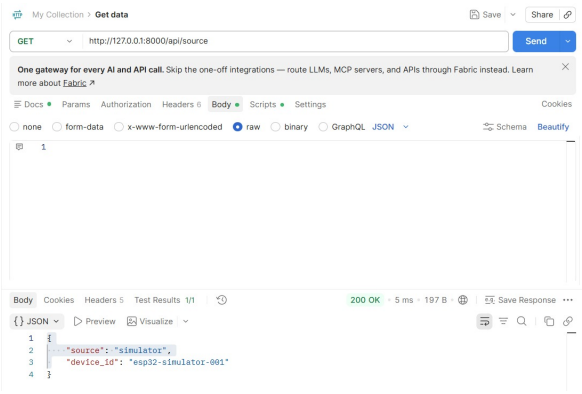
Método	**Rota**	**Função**
GET	`/health`	Verificação do funcionamento da API.
POST	`/api/telemetry`	Recebimento das informações enviadas pelo ESP32.
GET	`/api/telemetry/latest`	Consulta das informações mais recentes.
GET	`/api/actuators/led`	Consulta do estado solicitado para o LED.
POST	`/api/actuators/led`	Alteração do estado do LED através da interface.

A utilização dos métodos GET e POST permitiu implementar tanto operações de consulta quanto operações de envio e alteração de informações. Dessa maneira, foi possível estabelecer a comunicação necessária para aquisição e atuação utilizando o mesmo padrão REST.

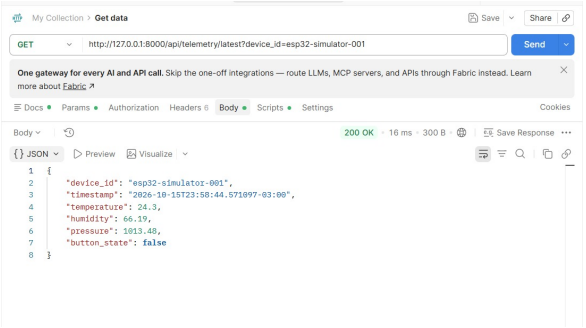
Durante os testes, as respostas HTTP também foram utilizadas para verificar o resultado das requisições. Isso foi importante principalmente durante a integração com o ESP32 físico, pois permitiu diferenciar falhas de conexão de requisições recebidas corretamente pelo servidor.



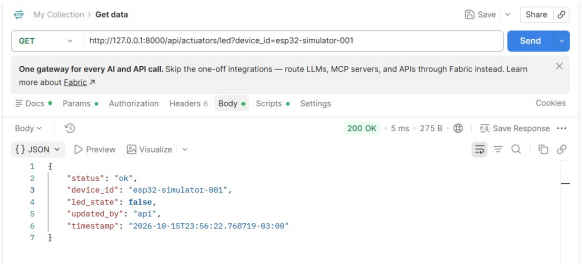
(a) Requisição POST para controle do LED.



(b) Requisição GET para consulta da fonte ativa.



(c) Requisição GET para consulta da telemetria.



(d) Requisição GET para o estado do LED.

Figura 5: Testes das principais rotas da API REST realizados através do Postman.

4.3 Construção da interface Node-RED

O Node-RED foi utilizado como interface principal para visualização e controle do sistema. A Figura 6 apresenta o fluxo desenvolvido durante o trabalho.

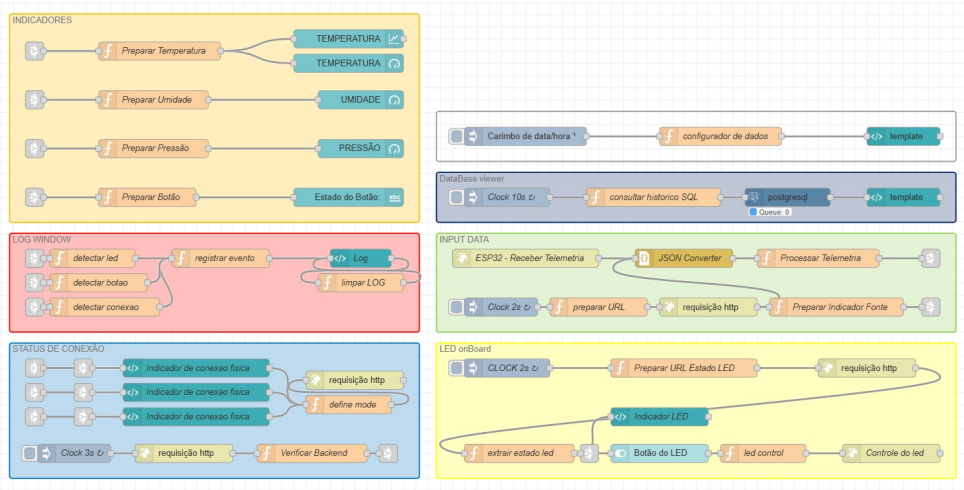


Figura 6: Fluxo desenvolvido no Node-RED para comunicação e supervisão do ESP32.

No dashboard foram criados elementos destinados à apresentação das variáveis provenientes do BME280. Dessa forma, é possível acompanhar os valores de temperatura, umidade relativa e pressão atmosférica obtidos pelo ESP32.

Além dos valores do sensor, também é apresentado o estado do botão físico. Quando ocorre seu acionamento, a alteração é enviada através da comunicação e apresentada no dashboard, permitindo verificar o funcionamento de uma entrada digital do dispositivo.

Para demonstrar a comunicação no sentido contrário, também foi implementado o controle do LED. Através da interface, o usuário pode solicitar que o atuador seja ligado ou desligado. O comando é enviado através da API e posteriormente recebido pelo ESP32.

Também foram adicionados indicadores para auxiliar na identificação do estado da comunicação e do dispositivo. Dessa maneira, além dos valores das variáveis, o usuário consegue verificar visualmente o estado do LED e se o dispositivo encontra-se comunicando corretamente.

4.3.1 Entrada dos dados

A entrada e o processamento dos dados no Node-RED são realizados pelo grupo de nós denominado *INPUT DATA*. Nesse grupo, foi implementado um nó *HTTP IN*, identificado como "ESP32 - Receber Telemetria", configurado para receber requisições do tipo POST por meio da rota `/api/telemetry`. Os dados recebidos são estruturados em formato JSON, contendo informações como temperatura, umidade, pressão atmosférica, estado do botão e identificação do dispositivo.

Além da entrada por POST, o fluxo possui uma consulta periódica à API REST. Um nó temporizador, configurado com intervalo de dois segundos, inicia a requisição HTTP para obtenção dos dados mais recentes. A resposta é encaminhada ao nó "JSON Converter" e, posteriormente, à função "Processar Telemetria", responsável por separar e converter os valores recebidos antes de encaminhá-los aos demais grupos do fluxo.

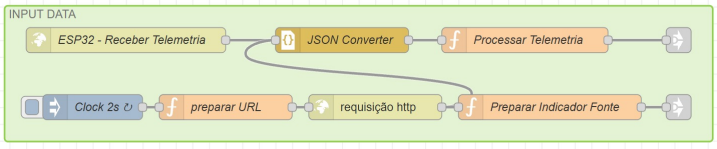


Figura 7: Captura de tela do grupo de entrada de dados no Node-Red.

4.3.2 Exposição dos dados obtidos

Após a chegada e o tratamento dos dados no *flow* do Node-RED, estes são encaminhados, por meio de nós *link*, ao grupo denominado *INDICADORES*. Nesse grupo, blocos do tipo *function* realizam a separação e preparação individual das variáveis de temperatura, umidade, pressão e estado do botão.

Cada função atribui ao `msg.payload` o valor correspondente à variável que será apresentada. Em seguida, os dados de temperatura, umidade e pressão são encaminhados aos respectivos elementos *gauge* do dashboard, enquanto o estado do botão é convertido para uma indicação textual, permitindo identificar se este se encontra pressionado ou não. Dessa forma, os dados recebidos são organizados e apresentados individualmente na interface de monitoramento.

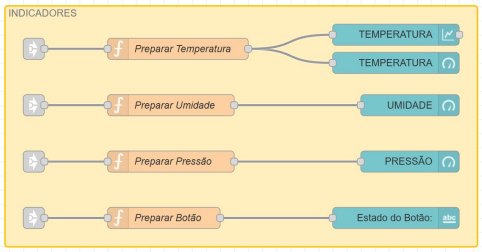
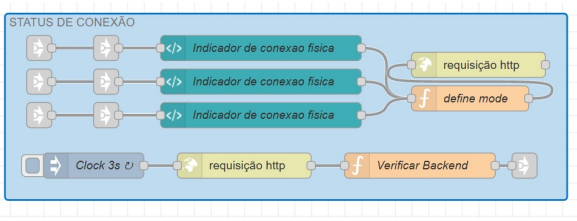


Figura 8: Captura de tela do grupo de indicares de dados no Node-Red.

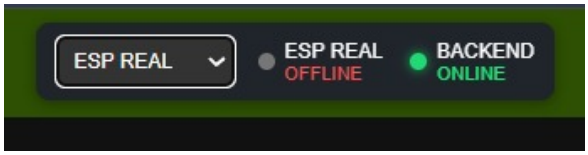
4.3.3 Status de conexão

O status de conexão é verificado periodicamente pelo Node-RED por meio do grupo *STATUS DE CONEXÃO*. Um nó temporizador inicia a verificação em intervalos regulares, realizando uma requisição HTTP à API REST de acordo com a fonte de dados selecionada no sistema.

A resposta da requisição é analisada por blocos *function*, que verificam se houve comunicação com o dispositivo ativo. A partir desse resultado, o fluxo determina o estado da conexão e atualiza as informações utilizadas pelo dashboard.



(a) Grupo de Status de conexão no Node-RED.



(b) Seleção da fonte de dados e indicador do Status de conexão.

Figura 9: Implementação e visualização do Status de conexão no Node-RED.

4.3.4 Dashboard construído



Figura 10: Dashboard principal desenvolvido no Node-RED.



Figura 11: Visualização complementar do dashboard desenvolvido no Node-RED.

A partir dos testes realizados foi possível observar tanto a atualização dos valores provenientes do sensor quanto a resposta do ESP32 aos comandos enviados pelo dashboard. Com isso, foi verificado o funcionamento da comunicação REST nos dois sentidos necessários para o projeto.

4.4 Realização do OTA

Além da comunicação REST, outro requisito do trabalho foi a implementação da programação remota do ESP32 utilizando OTA (*Over-the-Air*).

A utilização desse recurso permite atualizar o firmware através da própria rede Wi-Fi. Dessa maneira, após a configuração inicial, não é necessário conectar fisicamente o ESP32 ao computador sempre que uma nova versão do programa precisar ser enviada.

Para implementação foi utilizada a biblioteca `ArduinoOTA`. Durante a inicialização do firmware são configurados os parâmetros necessários para identificação e autenticação do dispositivo na rede.

No firmware foi definido um *hostname* para facilitar a identificação do dispositivo. Também foi configurada uma senha para impedir que a atualização seja realizada sem autenticação.

Durante o processo de OTA também foram configuradas funções responsáveis por indicar o início da atualização, seu progresso, finalização e possíveis erros. Essas informações permitem acompanhar a transferência e verificar se o novo firmware foi recebido corretamente.

Após a configuração inicial foram realizados testes de atualização através da rede. A Figura 12 apresenta um dos testes realizados com sucesso.

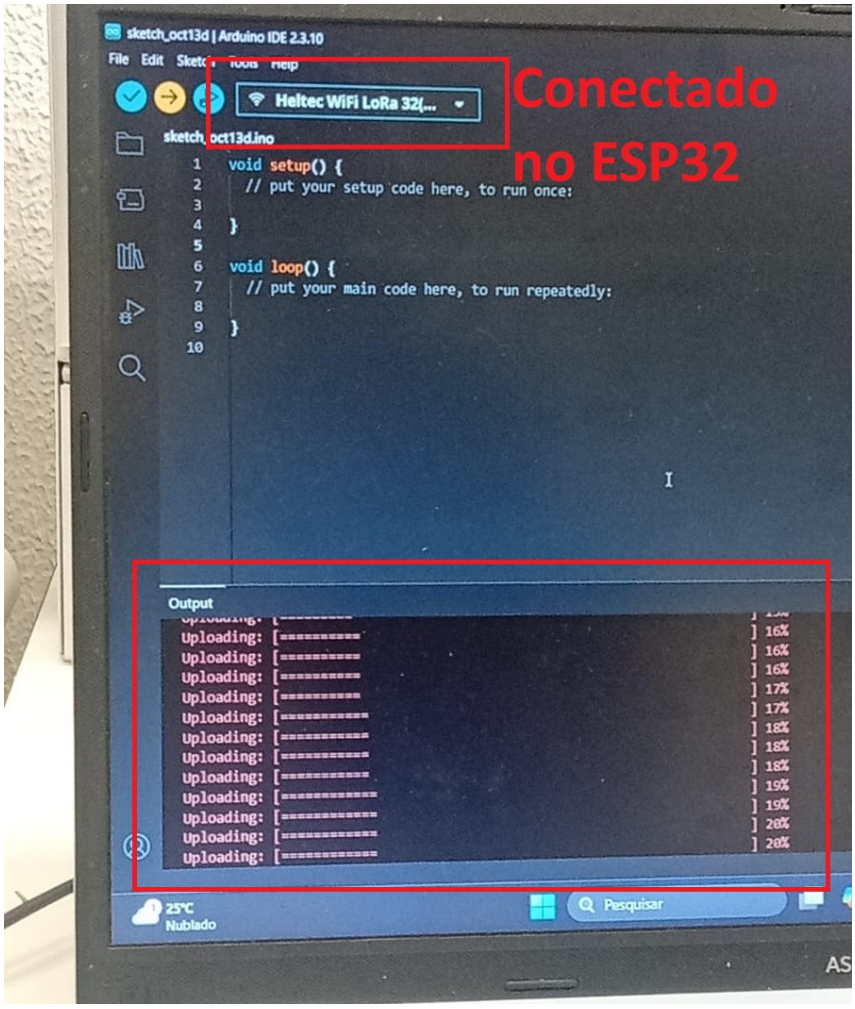


Figura 12: Atualização remota do firmware através de OTA realizada com sucesso.

A implementação do OTA se mostrou útil durante o próprio desenvolvimento do trabalho, uma vez que alterações posteriores no firmware puderam ser enviadas sem a necessidade de conectar novamente o cabo de programação ao ESP32.

4.5 Desenvolvimentos extras

Após a implementação dos requisitos principais do trabalho, foram desenvolvidos recursos adicionais com o objetivo de ampliar o sistema. Esses recursos não são necessários para realizar a comunicação REST entre o ESP32 e o Node-RED nem para o funcionamento do OTA, sendo portanto considerados desenvolvimentos extras.

4.5.1 Simulador do ESP32

Como primeiro desenvolvimento adicional, foi criado um simulador do ESP32 utilizando Python. Sua finalidade foi permitir a execução de uma fonte virtual de dados sem depender permanentemente do dispositivo físico.

O simulador reproduz parte do comportamento do ESP32, gerando valores correspondentes às variáveis ambientais e mantendo estados virtuais para o botão e para o LED. Para diferenciá-lo do dispositivo real, foi utilizado um identificador próprio.

A utilização do simulador também facilitou alguns testes durante o desenvolvimento, principalmente quando o ESP32 físico não estava conectado.

A Figura 13 apresenta o simulador em execução.

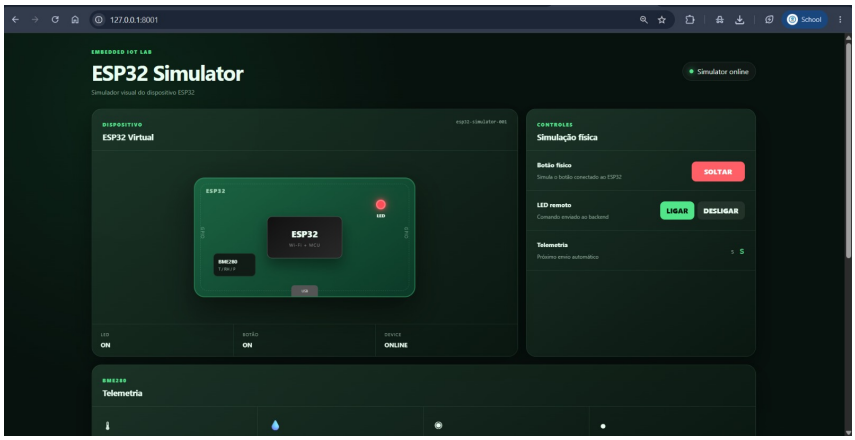


Figura 13: Simulador do ESP32 desenvolvido como funcionalidade adicional.

Para permitir a utilização tanto do dispositivo real quanto do simulador, foi adicionada uma seleção de fonte. O ESP32 físico é identificado como `esp32-001`, enquanto o simulador utiliza `esp32-simulator-001`. Assim, o usuário pode definir qual das duas fontes deseja utilizar durante os testes.

4.5.2 Interface Web

Também foi desenvolvida uma interface Web independente do Node-RED. Essa interface foi construída utilizando HTML, CSS e JavaScript e permite acompanhar as informações do sistema através de um navegador.

A aplicação apresenta os valores obtidos pelos sensores, informações relacionadas ao dispositivo e o estado do atuador. Também foi implementado o controle do LED através da própria interface.

A Figura 14 apresenta a interface desenvolvida.

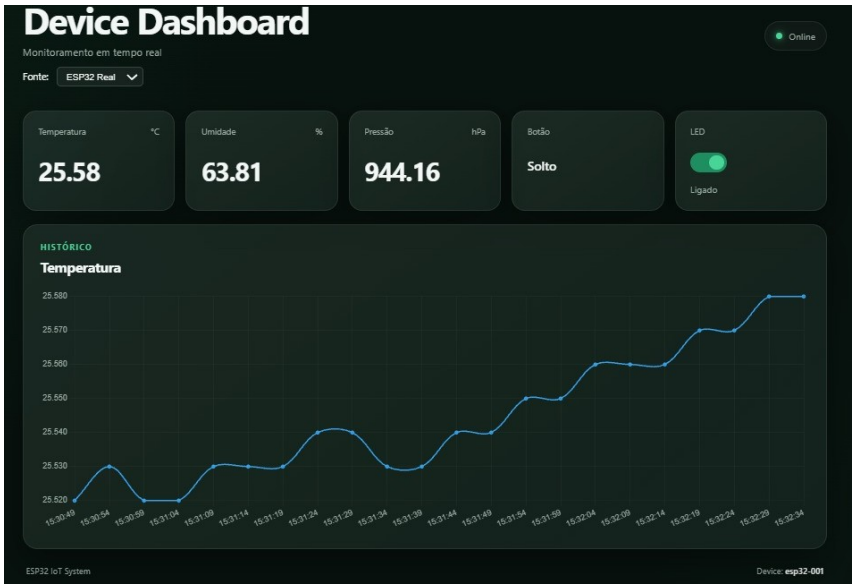


Figura 14: Interface Web desenvolvida como funcionalidade adicional.

A interface Web não substitui o dashboard desenvolvido no Node-RED. Sua implementação foi realizada como uma funcionalidade adicional, demonstrando que os mesmos dados podem ser utilizados por outras formas de visualização.

4.5.3 Tela de LOGS no Node-RED

Foi desenvolvido um LOG de atividades no Node-RED, essa função é útil para rastrear conexões, comandos e possíveis problemas presentes na planta. Para a implementação foram utilizados nós *link*, responsáveis por receber informações provenientes de diferentes partes do *flow* e encaminhá-las para o bloco responsável pelo processamento dos registros.

As mensagens recebidas são tratadas e padronizadas antes de serem apresentadas na interface. Cada registro contém informações referentes ao evento ocorrido, permitindo acompanhar alterações no estado da conexão, comandos enviados ao LED, mudanças no estado do botão e demais atividades relevantes durante a execução do sistema.

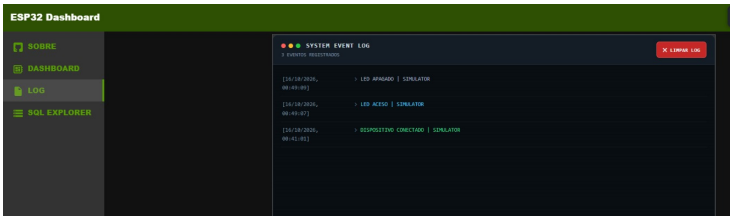


Figura 15: Captura de tela da página de LOG no Node-Red.

4.5.4 Banco de dados PostgreSQL

Como outro desenvolvimento adicional, foi implementado um banco de dados PostgreSQL para armazenamento das informações recebidas pelo sistema.

O objetivo dessa implementação foi permitir que as informações não fossem apenas apresentadas em tempo real, mas também armazenadas para consultas posteriores.

Foram utilizadas duas tabelas principais. A tabela `sensor_logs` é responsável pelo armazenamento das informações relacionadas às medições e ao botão, enquanto a tabela `actuator_status` registra alterações relacionadas ao estado do LED.

A Figura 16 apresenta registros relacionados às informações adquiridas.

	id [PK] integer	timestamp timestamp with time zone	device_id character varying (100)	led_state boolean	updated_by character varying (100)
464	464	2026-10-10 11:35:53.554304-...	esp32-simulator-001	true	node-red
465	465	2026-10-10 11:35:55.556982-...	esp32-simulator-001	true	node-red
466	466	2026-10-10 11:35:57.573627-...	esp32-simulator-001	true	node-red
467	467	2026-10-10 11:35:59.569714-...	esp32-simulator-001	true	node-red
468	468	2026-10-10 11:36:01.568982-...	esp32-simulator-001	true	node-red
469	469	2026-10-10 11:36:03.580864-...	esp32-simulator-001	true	node-red
470	470	2026-10-10 11:36:05.58184-03	esp32-simulator-001	true	node-red
471	471	2026-10-10 11:36:07.594671-...	esp32-simulator-001	true	node-red
472	472	2026-10-10 11:36:09.608365-...	esp32-simulator-001	true	node-red

Total rows: 13180 Query complete 00:00:00.556

Figura 16: Registros armazenados na tabela de telemetria do PostgreSQL.

A Figura 17 apresenta registros relacionados ao atuador.

Data Output | Messages | Notifications

Showing rows: 1 to 1000

	id [PK] integer	timestamp timestamp with time zone	device_id character varying (100)	sensor_name character varying (100)	sensor_value double precision	button_state boolean
1	1	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.12	false
2	2	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.65	false
3	3	2026-10-06 14:45:52.256704-...	esp32-simulator-001	pressure	1013.03	false
4	4	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.15	false
5	5	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.46	false
6	6	2026-10-06 14:45:52.256704-...	esp32-simulator-001	pressure	1013.14	false
7	7	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.22	false
8	8	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.54	false
9	9	2026-10-06 14:45:52.256704-...	esp32-simulator-001	pressure	1013.02	false

Total rows: 27087 | Query complete 00:00:00.377

Figura 17: Registros relacionados ao estado do LED armazenados no PostgreSQL.

A implementação do banco de dados possibilitou manter um histórico das informações produzidas durante a execução, ampliando as possibilidades de análise sem alterar o funcionamento principal da comunicação entre ESP32 e Node-RED. Além disso, foi desenvolvida uma página exclusiva para filtrar, validar e verificar o histórico do SQL no Node-red, de modo a facilitar a integração do usuário com as funcionalidades aplicadas.

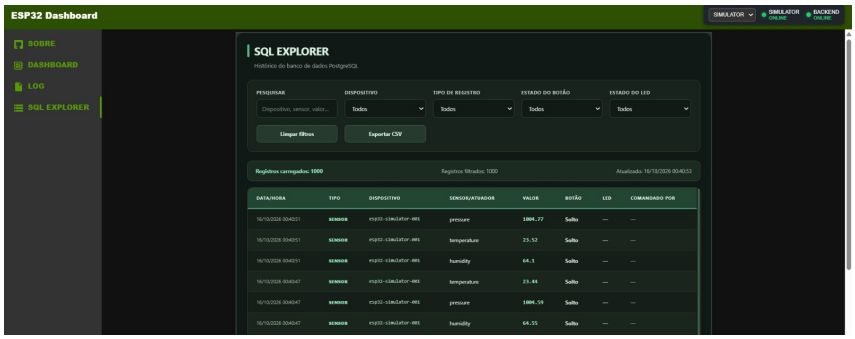


Figura 18: Captura de tela da página do Node-red que permite leitura personalizável do Node-Red.

Dessa forma, o simulador, a interface Web e o banco PostgreSQL foram utilizados para complementar o projeto, mas permaneceram independentes dos requisitos fundamentais relacionados à comunicação REST e ao OTA.

5 Conclusão

O desenvolvimento deste trabalho permitiu implementar um sistema completo de comunicação entre um ESP32 e diferentes aplicações através de uma API REST HTTP. A partir do sensor BME280 foi possível realizar a leitura das variáveis ambientais e enviar essas informações através da rede Wi-Fi, enquanto o LED pôde ser controlado de maneira remota.

A utilização do Node-Red permitiu reunir em uma única interface os dados obtidos, gráficos, estado do botão, controle do LED e informações sobre a conexão do dispositivo. Além disso, a implementação do OTA possibilitou realizar novas programações do ESP32 através da rede, diminuindo a dependência da conexão física com o computador após a primeira gravação.

Durante o desenvolvimento também foram implementadas algumas funcionalidades extras, principalmente o simulador do ESP32, a interface Web e o banco de dados PostgreSQL. O simulador permitiu desenvolver grande parte do sistema mesmo sem o equipamento físico conectado, enquanto o banco de dados possibilitou manter um histórico das medições realizadas.

A criação de uma opção para selecionar entre o simulador e o ESP32 real também facilitou bastante os testes, pois ambos utilizam praticamente a mesma estrutura de comunicação com a API. Dessa forma, foi possível substituir a fonte dos dados sem precisar modificar manualmente todas as requisições utilizadas pelo sistema.

De forma geral, os resultados obtidos atenderam os principais objetivos propostos para o trabalho, sendo possível realizar a aquisição, transmissão, armazenamento e visualização dos dados, além do controle remoto de um atuador. A estrutura desenvolvida também permite que futuramente outros sensores ou atuadores sejam adicionados, utilizando a mesma arquitetura de comunicação já implementada.